Ad Soyad: _____	**Tarih:** _____
Bölüm 8 – Ünite 5 – *Release Derleme ve Dağıtım – Cevap Anahtarı*	

Bölüm 1: Boşluk Doldurma – Cevaplar

1. **CMAKE_BUILD_TYPE**
2. **-g**, **NDEBUG**
3. **Link-Time Optimization**, **CMAKE_INTERPROCEDURAL_OPTIMIZATION**
4. **macdeployqt**, **windeployqt**, **linuxdeployqt**
5. **CMakePresets.json**

Bölüm 2: Eşleştirme – Cevaplar

1→c, 2→d, 3→a, 4→b, 5→e

Bölüm 3: Doğru/Yanlış – Cevaplar

1. **D** – `NDEBUG` tanımlı olduğunda `assert` makroları boşa çıkarılır.
2. **Y** – Kullanıcının makinesinde Qt kurulu olmayabilir; deployment araçlarıyla bağımlılıklar paketlenmelidir.
3. **D** – Ayrı build klasörleri ara dosya çakışmalarını önler.
4. **D** – `install_name_tool` framework referanslarını bundle-içi `@rpath`'a yönlendirir.
5. **Y** – LTO derleme süresini *uzatır* (binary optimizasyonu için); debug build'de kapalıdır.

Bölüm 4: Kısa Cevap – Cevaplar

1. Örnek karşılaştırma tablosu:

Özellik	**Debug**	**Release**
Optimizasyon	`-O0` (kapalı)	`-O2` veya `-O3`
Debug sembolleri	`-g` (dahil)	Çıkarılmış / ayrı dosyada
`NDEBUG` makrosu	Tanımsız	Tanımlı (`assert` boşa)
Hata ayıklama	Kolay (adım adım)	Zor
Binary boyutu	Büyük	Küçük

2. Self-contained paket, kullanıcının sisteminde ek bir Qt kurulumu gerektirmeden çalışabilen, gerekli tüm dinamik kütüphaneleri içinde barındıran dağıtım paketidir. `macdeployqt`'nin üç işi: (1) Qt framework'lerini `Contents/Frameworks/` altına kopyalar; (2) binary ve framework'lerdeki `@rpath` referanslarını `install_name_tool` ile yeniden yazar; (3) `-dmg` parametresi ile dağıtılabilir DMG dosyası üretir.
3. Avantajlar: (a) Versiyon tek yerden yönetilir, tüm altbileşenlere (Info.plist, pencere başlığı, `About` diyaloğu) aktarılabilir. (b) Versiyon makrosu C++ koduna `target_compile_definitions` ile geçirilebilir; çıktıda görünen sürüm, derlenen binary ile garanti aynıdır. (c) Semantic versioning (X.Y.Z) otomatik olarak `PROJECT_VERSION_MAJOR/MINOR/PATCH` değişkenlerine ayrılır.

Bölüm 5: Kod/Komut Okuma – Cevaplar

Satır 1: Kaynak dizini `.` ile, çıktı dizini `build-release` ile CMake yapılandırması yapılır; build tipi `Release` olarak ayarlanır.

Satır 2: `build-release` dizininde paralel (çok çekirdek) derleme yapılır.

Satır 3: Derleme çıktı dizinine geçilir.

Satır 4: `obys_v5.app` bundle'ını self-contained yapar; Qt framework'lerini kopyalar ve sonunda bir `.dmg` üretir.

Satır 5: `.app` bundle'ı "Developer ID" sertifikasıyla derin (tüm iç binary'ler) kod imzalaması yapar; macOS Gatekeeper'ın güvenmesi için gereklidir.

Bölüm 6: Uygulama – Cevaplar

1. Örnek dağıtım adımları:
   (a) CMake ile Release build yapılandırması (`CMAKE_BUILD_TYPE=Release`).
   (b) `cmake -build build-release -parallel` ile binary üretimi.
   (c) `windeployqt obys_v5.exe` ile Qt DLL'lerinin binary yanına kopyalanması.
   (d) Versiyon bilgisi, ikon, gerekli ek kaynaklar kontrolü.
   (e) Inno Setup script'i (`.iss`) ile installer'ın derlenmesi.
   (f) Kod imzalama (signtool) ile güvenilirlik.
   (g) Üretilen `setup.exe`'nin test edilmesi ve kullanıcıya dağıtılması.

2. Preset örneği:

```
{
  "version": 3,
  "configurePresets": [
    {
      "name": "release",
      "displayName": "OBYS Release",
      "binaryDir": "${sourceDir}/build-release",
      "cacheVariables": {
        "CMAKE_BUILD_TYPE": "Release",
        "CMAKE_INTERPROCEDURAL_OPTIMIZATION": "ON"
      }
    }
  ]
}
```

3. Inno Setup örneği:

```
[Files]
Source: "build-release\obys_v5.exe"; DestDir: "{app}"; Flags:
   ignoreversion
Source: "build-release\*"; DestDir: "{app}"; Flags: ignoreversion
   recursesubdirs createallsubdirs
```